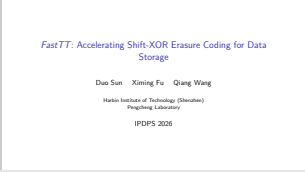
FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage

Duo Sun Ximing Fu Qiang Wang

Harbin Institute of Technology (Shenzhen)
Pengcheng Laboratory

IPDPS 2026

FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage



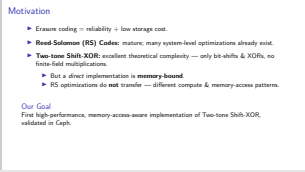
Good afternoon. I am Duo Sun, presenting FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage, joint work with my supervisors Prof. Ximing Fu and Prof. Qiang Wang at HIT Shenzhen. Two-tone is a Shift-XOR erasure code that replaces finite-field multiplications with bit-shifts and XORs. This talk is about turning that theoretical efficiency into a practical, high-throughput implementation inside Ceph.

Motivation

- Erasure coding = reliability + low storage cost.
- **Reed-Solomon (RS) Codes:** mature; many system-level optimizations already exist.
- **Two-tone Shift-XOR:** excellent theoretical complexity — only bit-shifts & XORs, no finite-field multiplications.
  - But a *direct* implementation is **memory-bound**.
  - RS optimizations do **not** transfer — different compute & memory-access patterns.

Our Goal

First high-performance, memory-access-aware implementation of Two-tone Shift-XOR, validated in Ceph.


Motivation

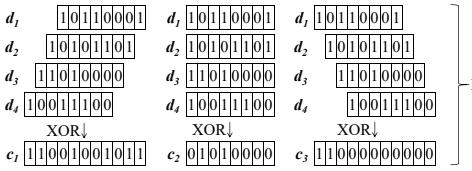
- Erasure coding = reliability + low storage cost.
- **Reed-Solomon (RS) Codes:** mature; many system-level optimizations already exist.
- **Two-tone Shift-XOR:** excellent theoretical complexity — only bit-shifts & XORs, no finite-field multiplications.
  - But a *direct* implementation is **memory-bound**.
  - RS optimizations do **not** transfer — different compute & memory-access patterns.

Our Goal

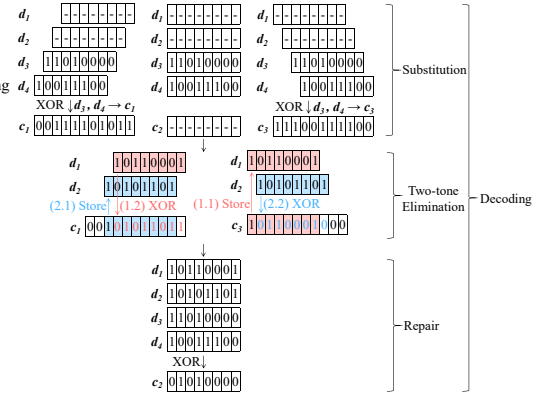
First high-performance, memory-access-aware implementation of Two-tone Shift-XOR, validated in Ceph.

Erasure coding delivers reliability at much lower storage cost than replication. RS Codes are the dominant choice and already heavily optimized. Our focus is Two-tone Shift-XOR: theoretically excellent, only shifts and XORs, no finite-field multiplications. The gap is on the implementation side: a direct Two-tone implementation is memory-bound, and RS-specific tricks do not transfer because the computation and memory-access patterns are fundamentally different. FastTT is the first high-performance, memory-access-aware implementation, validated in Ceph.

Two-tone Shift-XOR in One Slide



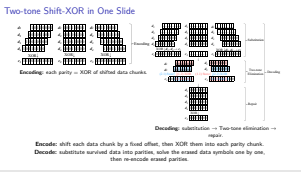
Encoding: each parity = XOR of shifted data chunks.



Decoding: substitution → Two-tone elimination → repair.

Encode: shift each data chunk by a fixed offset, then XOR them into each parity chunk.

Decode: substitute survived data into parities, solve the erased data symbols one by one, then re-encode erased parities.



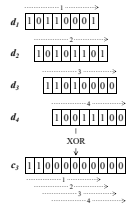
Here is Two-tone in one slide. The coding algorithm and its successive-solvability property were established in prior work, and our contribution is on the systems side. To encode, we split the data into k chunks and produce m parity chunks by XORing shifted versions of the data, with no finite-field multiplications anywhere. To decode, we first substitute the survived data into the survived parities, then run Two-tone elimination to solve the erased symbols one by one, and finally re-encode any erased parity chunks. The decoding order is predetermined by the code structure, so it is not searched at runtime. Now let us see why a direct implementation of this is still not fast enough.

Limitations (1/2): Encoding is Memory-Bound

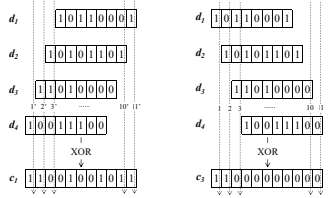
- **Redundant memory access & poor temporal locality**
  - Each parity chunk fully scanned *many* times.
  - Parity reads & writes repeat per data chunk.
- **Both traversal patterns are suboptimal**
  - *Horizontal:* extra parity writes.
  - *Vertical:* extra data reloads.

Bottleneck

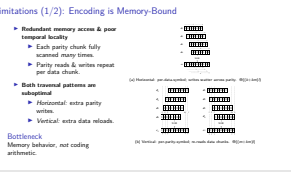
Memory behavior, *not* coding arithmetic.



(a) Horizontal: per-data-symbol; writes scatter across parity. $\Theta((k+km)l)$



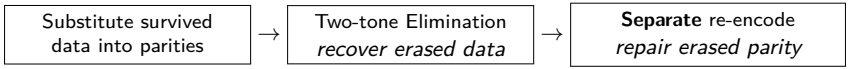
(b) Vertical: per-parity-symbol; re-reads data chunks. $\Theta((m+km)l)$



Two encoding bottlenecks. First, redundant memory access: the naive implementation scans each data chunk and performs a full pass over every parity chunk, so each parity is read and written k times. The cost is repeated memory traffic, not the XOR itself. Second, both traversal patterns are suboptimal. Horizontal has good data-read locality but inflates parity writes. Vertical writes each parity symbol once but repeatedly reloads data across k chunks. Both give traffic on the order of $k \cdot m \cdot l$. The bottleneck is memory behavior, not coding arithmetic.

Limitations (2/2): Complex Decoding Flow

- Naive decoding treats erased **data** and erased **parity** as two independent stages:



Problems

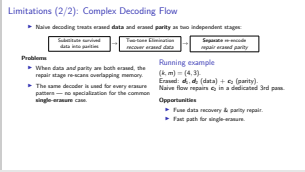
- When data *and* parity are both erased, the repair stage re-scans overlapping memory.
- The same decoder is used for every erasure pattern — no specialization for the common **single-erasure** case.

Running example

$(k, m) = (4, 3)$.
Erased: $\boldsymbol{d}_1, \boldsymbol{d}_2$ (data) + $\boldsymbol{c}_2$ (parity).
Naive flow repairs $\boldsymbol{c}_2$ in a dedicated 3rd pass.

Opportunities

- Fuse data recovery & parity repair.
- Fast path for single-erasure.


Limitations (2/2): Complex Decoding Flow

- Naive decoding treats erased **data** and erased **parity** as two independent stages:

Substitute survived data into parities → Two-tone Elimination recover erased data → Separate re-encode repair erased parity

Problems

- When data and parity are both erased, the repair stage re-scans overlapping memory.
- The same decoder is used for every erasure pattern — no specialization for the common **single-erasure** case.

Running example

$(k, m) = (4, 3)$.
Erased: d_1, d_2 (data) + c_2 (parity).
Naive flow repairs c_2 in a dedicated 3rd pass.

Opportunities

- Fuse data recovery & parity repair.
- Fast path for single-erasure.

The decoding side has two problems. First, naive Two-tone separates data recovery from parity repair. In the (4,3) example with d_1, d_2, and c_2 lost, the repair of c_2 is a dedicated third pass that re-reads all data chunks, creating redundant memory traffic. Second, the same decoder is applied to every erasure pattern, with no specialization for the common single-erasure case. These two observations motivate EPM and TPS.

FastTT Overview

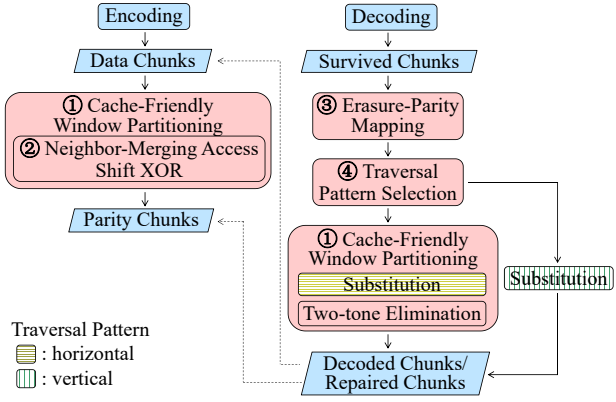
Design principle

Optimize Two-tone by **memory-access behavior**, not by algorithmic rewriting.

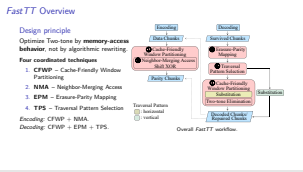
Four coordinated techniques

1. **CFWP** – Cache-Friendly Window Partitioning
2. **NMA** – Neighbor-Merging Access
3. **EPM** – Erasure-Parity Mapping
4. **TPS** – Traversal Pattern Selection

Encoding: CFWP + NMA.
Decoding: CFWP + EPM + TPS.



Overall *FastTT* workflow.



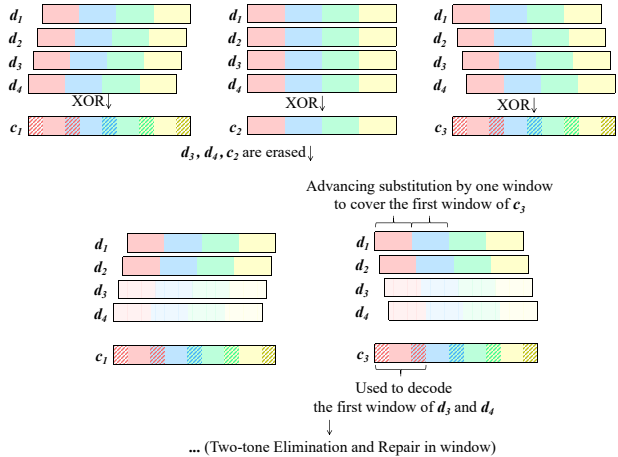
FastTT's design principle: optimize Two-tone by attacking memory-access behavior, not by changing the code construction. Four coordinated techniques: CFWP and NMA on the encoding side, EPM and TPS on the decoding side. The next four slides walk through each one.

Key Idea 1 – Cache-Friendly Window Partitioning

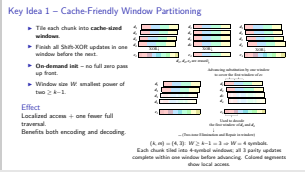
- Tile each chunk into **cache-sized windows**.
- Finish all Shift-XOR updates in one window before the next.
- **On-demand init** – no full zero pass up front.
- Window size W: smallest power of two $\geq k-1$.

Effect

Localized access + one fewer full traversal.
Benefits both encoding and decoding.



$(k, m) = (4, 3)$: $W \geq k-1 = 3 \Rightarrow W = 4$ symbols.
Each chunk tiled into 4-symbol windows; all 3 parity updates complete within one window before advancing. Colored segments show local access.



Throughout the key ideas I will use the (4,3) code as a running example, matching the figures. For CFWP: the minimum window size is $k - 1 = 3$, rounded up to the next power of two gives $W = 4$ symbols. Each chunk is tiled into 4-symbol windows, and all 3 parity updates for a window complete before moving to the next. The working set stays small and fits in cache. On-demand initialization removes one full zero-pass over the chunk. This benefits both encoding and decoding.

Key Idea 2 – Neighbor-Merging Access

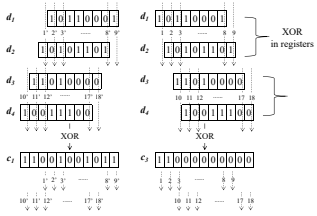
- XOR **two adjacent** data symbols in SIMD registers *first*.
- Then write the merged result to parity *once*.
- Keeps horizontal-style read locality; halves parity writes.

Horizontal	$\Theta((k+km)l)$
Vertical	$\Theta((m+km)l)$
Neighbor-Merging	$\Theta\big((k+\frac{km}{2})l\big)$

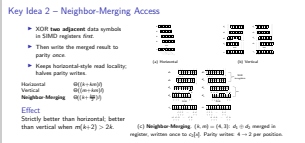
Effect

Strictly better than horizontal; better than vertical when $m(k+2) > 2k$.

(a) Horizontal (b) Vertical



(c) **Neighbor-Merging**. $(k, m) = (4, 3)$: $d_1 \oplus d_2$ merged in register, written once to $c_2[x]$. Parity writes: $4 \to 2$ per position.



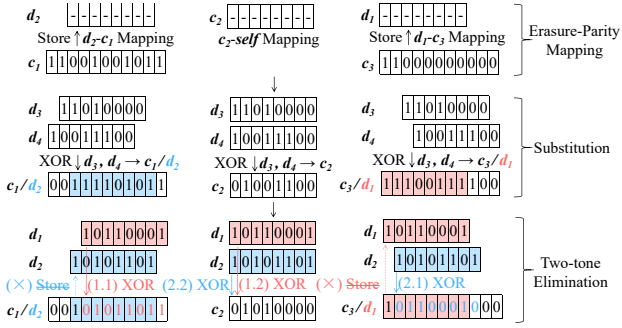
In the (4,3) code, c_2 has offset 0 for all data chunks, so d_1 and d_2 both contribute to $c_2[x]$ without any shift. Instead of writing to c_2 twice, we XOR $d_1[x]$ and $d_2[x]$ together in a SIMD register and write once. Parity writes drop from 4 to 2 per symbol position. The complexity table on the slide shows that neighbor-merging at $\Theta((k+km/2)l)$ is strictly better than horizontal and, in typical configurations, also better than vertical. We limit merging to pairs because AVX2 has a finite register budget and merging more chunks complicates offset alignment.

Key Idea 3 – Erasure-Parity Mapping

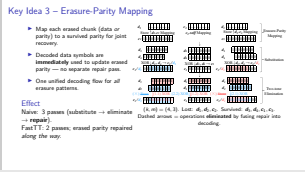
- Map each erased chunk (data *or* parity) to a survived parity for joint recovery.
- Decoded data symbols are **immediately** used to update erased parity — no separate repair pass.
- One unified decoding flow for *all* erasure patterns.

Effect

Naive: 3 passes (substitute → eliminate → **repair**).
FastTT: 2 passes; erased parity repaired *along the way*.



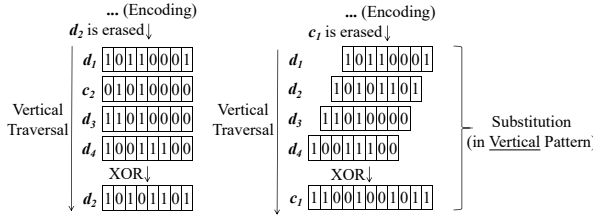
$(k, m) = (4, 3)$. Lost: $\boldsymbol{d}_1, \boldsymbol{d}_2, \boldsymbol{c}_2$. Survived: $\boldsymbol{d}_3, \boldsymbol{d}_4, \boldsymbol{c}_1, \boldsymbol{c}_3$.
Dashed arrows = operations **eliminated** by fusing repair into decoding.



In the (4,3) example with d_1, d_2, c_2 lost: naive decoding substitutes d_3, d_4 into c_1, c_3, runs elimination to recover d_1 and d_2, then performs a separate third pass to re-encode c_2 from scratch. That third pass re-reads all data, which is redundant. FastTT maps each erased chunk to a survived parity: $d_1 \to c_1$, $d_2 \to c_3$, $c_2 \to c_2$ itself. During elimination, every decoded symbol of d_1 or d_2 is immediately XORed into the c_2 buffer. By the time elimination finishes, c_2 is already repaired. The dashed arrows in the figure show the operations eliminated by this fusion.

Key Idea 4 – Traversal Pattern Selection

- Only **1 chunk** lost ⇒ each survived chunk read **once**, result written **once**.
- Vertical pattern: $\Theta((k+1)l)$. Horizontal: $\Theta((k+m)l)$ — extra parity reads/writes for the $m-1$ untouched parities.
- No merging needed: each data chunk contributes exactly once, so pairwise XOR adds no benefit.
- All accesses are **sequential & single-pass** ⇒ use SIMD **streaming stores** (bypass cache, avoid pollution).

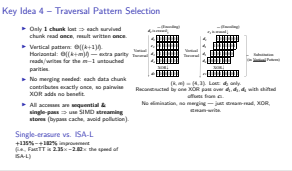


$(k, m) = (4, 3)$. Lost: $\boldsymbol{d}_2$ only.
Reconstructed by one XOR pass over $\boldsymbol{d}_1, \boldsymbol{d}_3, \boldsymbol{d}_4$ with shifted offsets from $\boldsymbol{c}_1$.

No elimination, no merging — just stream-read, XOR, stream-write.

Single-erasure vs. ISA-L

+135% – **+182%** improvement
(i.e., FastTT is **2.35**× – **2.82**× the speed of ISA-L)



When only one chunk is lost, each survived chunk is read exactly once and the result is written once. Vertical pattern costs $\Theta((k+1)l)$; horizontal would still touch the $m-1$ untouched parities, costing $\Theta((k+m)l)$. There is also no benefit from neighbor-merging: each data chunk contributes exactly one XOR, nothing to merge. Because all accesses are sequential and single-pass, we use SIMD streaming stores to bypass the cache entirely, avoiding pollution and maximizing bandwidth. In the (4,3) example with only d_2 lost, we stream-read d_1, d_3, d_4 with their shifted offsets from c_1, XOR them, and stream-write directly to d_2. No elimination needed. Result: single-erasure decoding is 135–182% faster than ISA-L, i.e., 2.35–2.82× its speed. Multi-erasure falls back to EPM.

Flagship Results – Throughput in Ceph

vs. Ceph production baselines

Intel i9-14900K · AVX2 · 2 MB chunks

- vs. ISA-L: +59.1% encode, +50.9% decode
- vs. Jerasure: +261% encode, +161% decode

	Encode (GB/s)			Avg. decode		
(k, m)	**FastTT**	ISA-L	Jera.	**FastTT**	ISA-L	Jera.
$(6, 2)$	34.7	24.4	17.0	60.8	31.6	29.5
$(6, 3)$	31.7	17.6	7.6	46.2	27.5	20.7
$(8, 3)$	31.4	17.0	7.1	46.9	24.9	17.7
$(10, 4)$	17.6	13.6	4.6	32.2	21.9	13.5

vs. academic SOTA (Cerasure)

- +13.2% encode on average
- +16.0% decode on average; peak +43.6%
- **Single-erasure: +135%–+182%** vs. ISA-L ($2.35\times - 2.82\times$ the speed).

	Encode (GB/s)		Avg. decode	
(k, m)	**FastTT**	Cerasure	**FastTT**	Cerasure
$(6, 2)$	48.0	44.2	67.6	59.7
$(6, 3)$	32.0	31.0	49.4	44.8
$(8, 3)$	32.8	28.7	48.7	43.9
$(10, 4)$	24.6	19.4	37.3	33.4

Avg. decode = averaged over erasure counts $1..m$, for both data- and parity-chunk losses.

Flagship Results – Throughput in Ceph

vs. Ceph production baselines

Intel i9-14900K · AVX2 · 2 MB chunks

- vs. ISA-L: +59.1% encode, +50.9% decode
- vs. Jerasure: +261% encode, +161% decode

	Encode (GB/s)			Avg. decode		
(k, m)	**FastTT**	ISA-L	Jera.	**FastTT**	ISA-L	Jera.
(6, 2)	34.7	24.4	17.0	60.8	31.6	29.5
(6, 3)	31.7	17.6	7.6	46.2	27.5	20.7
(8, 3)	31.4	17.0	7.1	46.9	24.9	17.7
(10, 4)	17.6	13.6	4.6	32.2	21.9	13.5

vs. academic SOTA (Cerasure)

- +13.2% encode on average
- +16.0% decode on average; peak +43.6%
- **Single-erasure: +135%–+182%** vs. ISA-L (2.35×–2.82× the speed).

	Encode (GB/s)		Avg. decode	
(k, m)	**FastTT**	Cerasure	**FastTT**	Cerasure
(6, 2)	48.0	44.2	67.6	59.7
(6, 3)	32.0	31.0	49.4	44.8
(8, 3)	32.8	28.7	48.7	43.9
(10, 4)	24.6	19.4	37.3	33.4

Avg. decode = averaged over erasure counts $1..m$, for both data- and parity-chunk losses.

All three baselines — ISA-L, Jerasure, and Cerasure — are RS-based libraries. Shift-XOR codes have no prior production-grade implementations in storage systems, so this is also the first time Two-tone is benchmarked head-to-head against optimized RS implementations under the same Ceph framework. Encoding-wise, FastTT consistently outperforms all baselines, including the academic state-of-the-art Cerasure, confirming that our memory-aware design effectively translates Two-tone's theoretical advantage into practical throughput.

For decoding, the average improvement is largely driven by the single-erasure case, where the TPS fast path uses streaming stores to deliver 2.35–2.82× the speed of ISA-L. As the number of erasures grows, the gap narrows: the single-erasure fast path no longer applies, and neighbor-merging cannot exploit the erased chunks during substitution because their content is unknown. Even so, FastTT still leads or ties the baselines on average across all configurations.

Flagship Results – Ablation on $(8, 3)$

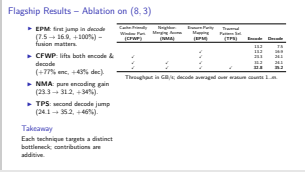
- **EPM**: first jump in *decode* ($7.5 \rightarrow 16.9$, $+100\%$) – fusion matters.
- **CFWP**: lifts both encode & decode ($+77\%$ enc, $+43\%$ dec).
- **NMA**: pure encoding gain ($23.3 \rightarrow 31.2$, $+34\%$).
- **TPS**: second decode jump ($24.1 \rightarrow 35.2$, $+46\%$).

Cache-Friendly Window Part. **(CFWP)**	Neighbor-Merging Access **(NMA)**	Erasure-Parity Mapping **(EPM)**	Traversal Pattern Sel. **(TPS)**	**Encode**	**Decode**
				13.2	7.5
		✓		13.2	16.9
✓		✓		23.3	24.1
✓	✓	✓		31.2	24.1
✓	✓	✓	✓	**32.8**	**35.2**

Throughput in GB/s; decode averaged over erasure counts $1..m$.

Takeaway

Each technique targets a distinct bottleneck; contributions are additive.



The ablation on (8,3) decomposes where the gains come from. EPM alone more than doubles decoding throughput, which validates our claim that fusing data recovery with parity repair removes a real bottleneck — the redundant memory traffic of the separate repair stage. CFWP then improves both sides because cache-sized windows benefit any traversal, encoding or decoding. NMA is encoding-specific by design: it merges adjacent data contributions before writing parity, which only applies on the encoding side, so decoding stays flat here. Finally TPS gives a second large decoding jump by routing single-erasure cases to the streaming-store fast path. The four techniques target different bottlenecks, and the table shows their contributions are additive — each one matters.

Summary and Future Research

One-sentence summary

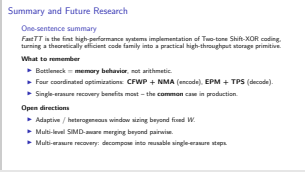
FastTT is the first high-performance systems implementation of Two-tone Shift-XOR coding, turning a theoretically efficient code family into a practical high-throughput storage primitive.

What to remember

- Bottleneck = **memory behavior**, not arithmetic.
- Four coordinated optimizations: **CFWP + NMA** (encode), **EPM + TPS** (decode).
- Single-erasure recovery benefits most – the **common** case in production.

Open directions

- Adaptive / heterogeneous window sizing beyond fixed W.
- Multi-level SIMD-aware merging beyond pairwise.
- Multi-erasure recovery: decompose into reusable single-erasure steps.


Summary and Future Research

One-sentence summary

FastTT is the first high-performance systems implementation of Two-tone Shift-XOR coding, turning a theoretically efficient code family into a practical high-throughput storage primitive.

What to remember

- Bottleneck = **memory behavior**, not arithmetic.
- Four coordinated optimizations: **CFWP** + **NMA** (encode), **EPM** + **TPS** (decode).
- Single-erasure recovery benefits most – the **common** case in production.

Open directions

- Adaptive / heterogeneous window sizing beyond fixed W.
- Multi-level SIMD-aware merging beyond pairwise.
- Multi-erasure recovery: decompose into reusable single-erasure steps.

FastTT is the first high-performance systems implementation of Two-tone Shift-XOR coding. The bottleneck is memory behavior, not arithmetic. Four coordinated techniques: CFWP and NMA for encoding, EPM and TPS for decoding. Single-erasure recovery, the dominant case in production, benefits most. Open directions: adaptive window sizing, multi-level SIMD merging, and decomposing multi-erasure patterns into reusable single-erasure steps. Thank you.

Thank you!

Questions & Discussion

Duo Sun, Ximing Fu, Qiang Wang

HIT (Shenzhen) · Pengcheng Laboratory

Thank you!
Questions & Discussion

Duo Sun, Ximing Fu, Qiang Wang
HIT (Shenzhen) · Pengcheng Laboratory

Thank you for listening. I am happy to discuss the implementation details, the memory-access modeling behind each of the four techniques, the comparison with Reed-Solomon libraries, or the implications for production storage deployments in Ceph.